International Islamic University Chittagong

Department of Computer Science and Engineering

GPT-Powered Threat Analyzer

with Real-Time Network Monitoring and Explainable Threat Analysis using Retrieval-Augmented Generation

A Project Report

Course Name: Computer Security Lab

Course Code: CSE-4744

Submitted By:

1. Mohammad Ibrahim Abdullah [C231165]
2. Mostafa Ahnaf Hossain [C231171]
3. Mohammed Junaid Mahmud [C231173]
4. Md. Mohiul Islam Miraz [C231197]

Supervised By:

Hasan Shorif

Adjunct Lecturer, Department of CSE, IIUC

_____

Approval of the Supervisor

Table of Contents

Abstract

Traditional network intrusion detection systems such as Snort and Suricata rely almost entirely on static, signature-based rules. They are fast and precise against known attack patterns, but they struggle with novel or subtly disguised behaviour, and the alerts they produce are often just a rule identifier or a CVE reference that still requires a trained analyst to interpret. This project addresses that gap by building a hybrid Threat Analyzer System (TAS) that combines classical, rule-based network triage with a modern Large Language Model for explanation and reasoning.

The system captures live network packets using Scapy, extracts structured metadata such as IP addresses, ports, protocol, TCP flags and payload size, and passes every packet through a lightweight heuristic triage engine. Only packets that the triage engine flags as suspicious are escalated to the AI layer. At that stage, a Retrieval-Augmented Generation (RAG) pipeline built on ChromaDB retrieves relevant MITRE ATT&CK technique descriptions from a local knowledge base, and this context is combined with the packet metadata and sent to the GPT-OSS 120B language model, accessed through the Groq Cloud API, for final classification and a human-readable explanation.

The result is delivered through a FastAPI backend and a Streamlit-based Security Operations Centre (SOC) dashboard, with all alerts persisted in a SQLite database for later review. The report that follows documents the motivation, architecture, implementation, and evaluation of this system, and compares it against traditional signature-based IDS tools to highlight where an LLM-assisted approach adds genuine value, and where it still falls short.

1. Introduction

1.1 Background

Network intrusion detection has traditionally been the job of signature-based engines. Tools like Snort and Suricata compare live traffic against a large, curated database of attack signatures and fire an alert when a match is found. This approach is fast enough to run at wire speed and produces very few false positives, but it is fundamentally

reactive: an attack has to be seen and catalogued before a signature can exist for it. It is also unfriendly to newcomers, because an alert is usually nothing more than a rule ID and a short technical description, leaving the interpretation of severity and next steps entirely to the analyst.

At the same time, Large Language Models have become capable enough to read structured technical data and produce a coherent, context-aware explanation of what that data means. This project explores whether an GPT, used carefully and only where it is actually needed, can sit on top of a conventional detection pipeline and turn a bare rule match into an explanation that a junior analyst, or even a non-specialist, can act on immediately.

1.2 Problem Statement

Small and medium-sized networks rarely have a dedicated, experienced Security Operations Centre. When an intrusion detection tool raises an alert, the person looking at it often does not have the background to judge whether the alert is a real threat, a false positive, or a low-priority anomaly. Existing open-source TAS tools are excellent at detection but weak at communication. There is a genuine need for a system that keeps the speed and reliability of rule-based detection while adding an explanation layer that is accurate, grounded in real threat-intelligence data, and written in plain language.

1.3 Objectives

- Capture live network traffic and extract meaningful packet-level features in real time.
- Apply a fast, deterministic heuristic engine to triage packets before any AI processing takes place.
- Use Retrieval-Augmented Generation to ground the language model's output in verified MITRE ATT&CK knowledge, reducing hallucination.
- Use the GPT-OSS 120B model, accessed through the Groq Cloud API, to classify flagged traffic and generate a clear, human-readable explanation with a recommended action.

- Persist every alert with its full context in a local database, and present it through a live, easy-to-read dashboard.
- Compare the resulting system, honestly and quantitatively, against traditional signature-based IDS tools.

1.4 Scope of the Project

The current implementation is designed to run on a single machine and monitor a single network interface, such as a laptop's Wi-Fi or Ethernet adapter. It is intended as an educational and demonstrative prototype for a Security-focused academic project rather than a production perimeter defence system, and its limitations, particularly around throughput and multi-interface support, are discussed in Section 9.

2. Related Work: Traditional TAS Tools

Signature-based intrusion detection systems such as Snort and Suricata remain the industry standard for perimeter defence because of their speed and the maturity of their rule ecosystems. They inspect packets against thousands of known attack signatures and can operate comfortably at multi-gigabit line rates. Their main limitation is that they are only as good as their signature database: an attack that does not match an existing rule, including most zero-day exploits, will pass through undetected.

This project takes a different, complementary approach. Instead of trying to out-perform Snort or Suricata on raw throughput or signature coverage, it focuses on the layer that traditional tools leave to the human analyst: interpretation and explanation. The table below summarises how the two approaches differ.

Criteria	**This GPT-TAS**	**Snort / Suricata**
Detection Method	Heuristic rules + AI classification	Signature-based rules
Explainability	Natural-language explanation for every alert	Rule ID and CVE reference only
Zero-Day Detection	Possible, through behavioural analysis	Limited; requires an existing signature
Setup Complexity	Moderate (Python stack + API key)	Complex (large rule-set management)

Criteria	This GPT-TAS	Snort / Suricata
Throughput	Roughly 100 packets/sec (LLM is the bottleneck)	10 Gbps and above, at wire speed
False Positives	Moderate; the AI can reason about context	Low; signatures are precise
Customisation	Prompt engineering and knowledge-base updates	Writing new Snort/Suricata rules
Best Suited For	SOC augmentation and threat explanation	Production perimeter defence

In practice, the two approaches are not mutually exclusive. A realistic deployment would most likely run a high-throughput signature engine at the perimeter and use an LLM-assisted layer such as this one to enrich and explain the subset of alerts that need human attention.

3. System Architecture

The system is organised as a pipeline of loosely coupled components, each running in its own thread or async task so that a slow stage, most notably the LLM call, never blocks packet capture or the REST API. Figure 1 shows the overall architecture.

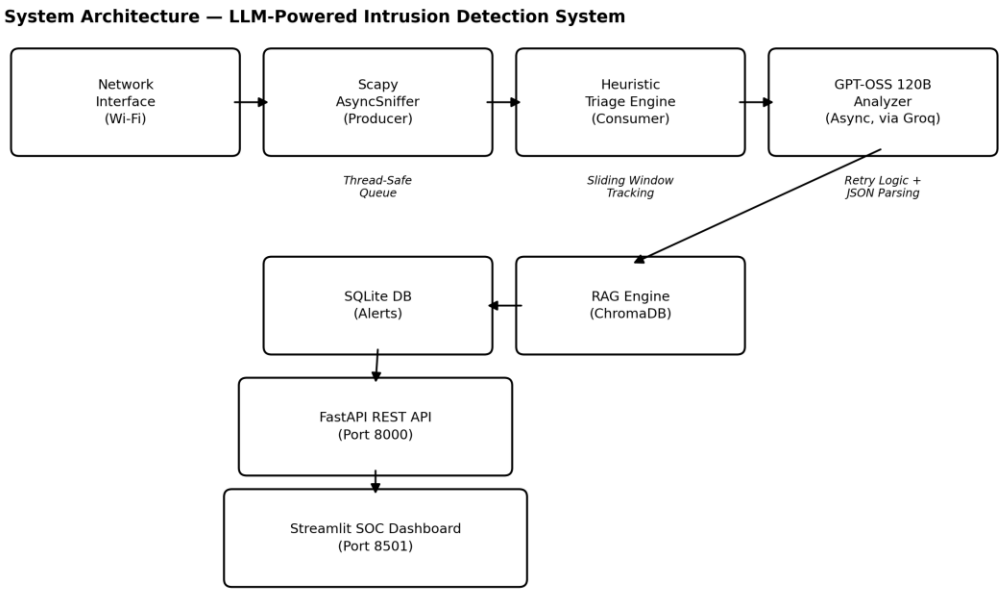

Figure 1: High-level system architecture of the GPT-Powered TAS.

3.1 Concurrency Model

Four independent execution contexts cooperate to keep the system responsive under load:

Thread / Task	Role	Pattern
Main (uvicorn)	FastAPI asynchronous request handling	Non-blocking
Sniffer Thread	Scapy AsyncSniffer packet capture	Producer
Triage Thread	Heuristic rule engine	Consumer
LLM Thread	GPT-OSS 120B asynchronous calls via Groq	Dedicated event loop

Packets move from the sniffer to the triage engine through a thread-safe queue, which decouples the rate of capture from the rate of processing and prevents a burst of traffic from stalling the network interface itself. Only packets that the triage engine marks as suspicious are ever placed on the queue that feeds the LLM thread, which is what keeps the AI layer from becoming a bottleneck under normal traffic conditions.

4. Methodology: How the System Works

The processing pipeline can be described as four stages: capture, triage, retrieval, and reasoning. Figure 2 traces a single packet through the full decision path, from the moment it is captured on the wire to the moment it either gets silently counted as normal traffic or surfaces as an explained alert on the dashboard.

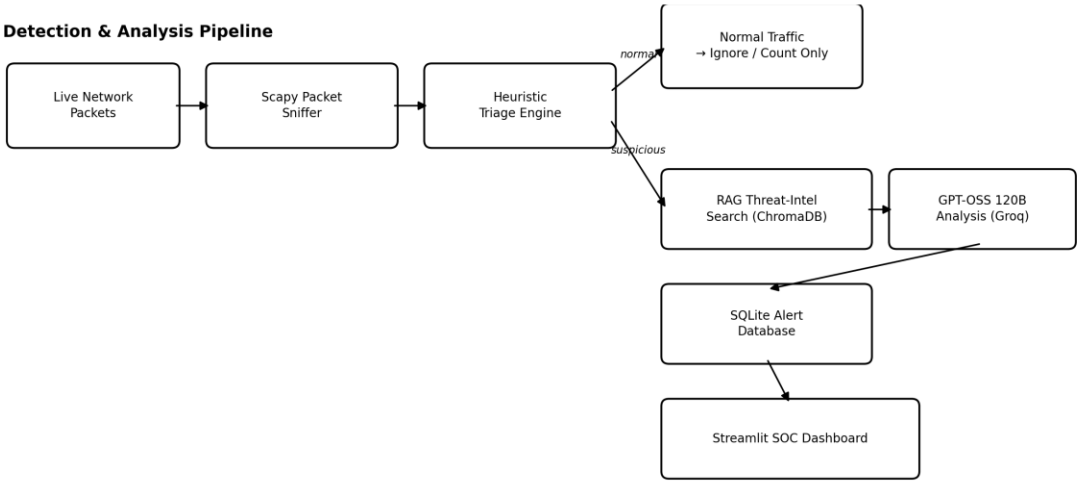

Figure 2: End-to-end detection and analysis pipeline.

4.1 Packet Capture and Feature Extraction

Live traffic is captured with Scapy's AsyncSniffer running in a background thread, so the FastAPI server is never blocked while packets are being read from the network interface. store=False is used deliberately so that raw packets are never retained in memory once their features have been extracted, which keeps the memory footprint bounded during long capture sessions. For every packet, the sniffer extracts:

- Source and destination IP address
- Source and destination port
- Protocol: TCP, UDP, or ICMP
- TCP flags, such as SYN, ACK and FIN
- Packet size and a limited payload preview
- DNS query information, where present
- A timestamp for correlation and sliding-window analysis

4.2 Heuristic Triage Engine

Sending every single packet to a language model would be slow, expensive, and would quickly exceed the free-tier request limits on Groq Cloud. Instead, a deterministic, rule-based triage engine inspects every packet first, using sliding-window counters to track behaviour per source IP over time rather than judging each packet in isolation. Only packets that trip one or more rules are escalated. The current rule set covers ten distinct attack patterns:

Rule	Description
SYN Scan	TCP SYN packets without a matching ACK, arriving in rapid succession
Port Sweep	A single source contacting many destination ports in a short window
ICMP Flood	An abnormally high rate of ICMP traffic from one source
NULL Scan	TCP packets sent with no flags set at all
XMAS Scan	TCP packets with the FIN, PSH and URG flags set together
FIN Scan	TCP packets with only the FIN flag set, a classic stealth-scan technique

Rule	Description
Suspicious Port	Traffic directed at a destination port on a known-bad-port list
DNS Tunnelling	Oversized DNS packets that may indicate data exfiltration
Large Payload	An unusually large payload on a non-web port
High Frequency	An excessive overall packet rate from a single source

Each rule contributes to a priority score, and auto-expiring entries in the sliding-window trackers keep memory usage bounded regardless of how long the sniffer has been running.

4.3 Retrieval-Augmented Generation (RAG)

RAG stands for Retrieval-Augmented Generation. Rather than relying purely on what the language model already knows, the system maintains a local threat-intelligence knowledge base (knowledge_base/threat_intel.json) containing more than twenty-five MITRE ATT&CK techniques, each with its technique ID, name, tactic, description, detection guidance, severity, and related keywords. This knowledge base is converted into vector embeddings and stored in ChromaDB, an embedded vector database.

When the triage engine raises flags such as PORT_SWEEP or SYN_FLOOD, the RAG engine queries ChromaDB for the techniques most relevant to those flags, for example T1046 (Network Service Scanning), T1498 (Network Denial of Service), or T1595 (Active Scanning), and injects the retrieved descriptions into the prompt sent to the language model. This division of labour keeps each component doing what it is best at: the triage engine decides what looked suspicious, the RAG engine supplies grounded cybersecurity knowledge, and the language model interprets the combined evidence and writes the final assessment. It also means the knowledge base can be corrected or expanded at any time without retraining or fine-tuning the model.

4.4 GPT-Based Threat Classification

Only packets that have already been flagged by the triage engine reach this stage. The flagged packet's metadata, together with the triage flags, a computed priority score, and the retrieved MITRE ATT&CK context, is formatted into a structured prompt and sent to the GPT-OSS 120B model through the Groq Cloud API. A professional SOC-

analyst system prompt instructs the model to reason about the evidence and return a strict JSON object, which is validated on receipt; if the response is malformed, the client retries up to three times with exponential backoff before giving up. A representative model response looks like this:

threat_level: "High" | confidence: 0.87 | attack_vector: "Possible reverse-shell connection" | mitre_technique: "T1059 - Command and Scripting Interpreter" | recommended_action: "Investigate the source host and block the connection if unauthorized."

The accompanying human_readable_explanation field is what ultimately appears on the dashboard, turning a raw set of triage flags into a short paragraph that a junior analyst can understand and act on without needing to look up rule numbers or technique IDs separately.

5. System Implementation

5.1 Module Breakdown

- **modules/sniffer.py — Packet Capture Engine**
    - Runs Scapy's AsyncSniffer in a background thread; extracts IP, ports, protocol, flags, and payload; uses store=False for memory efficiency; outputs to a thread-safe queue.
- **modules/triage.py — Heuristic Rule Engine**
    - Implements SlidingWindowTracker and PortTracker; evaluates the ten detection rules; computes a priority score; auto-expires stale entries to bound memory use.
- **modules/llm_client.py — GPT-OSS 120B Integration**
    - Holds the SOC-analyst system prompt; performs structured JSON parsing with validation; retries with exponential backoff; runs on its own dedicated asyncio event loop in a background thread.
- **modules/rag_engine.py — RAG Knowledge Base**
    - Wraps the embedded ChromaDB vector store; holds 25+ MITRE ATT&CK technique descriptions; performs contextual queries keyed on triage flags; returns the top-K matches for prompt injection.
- **database.py — SQLite Persistence**

- Async database operations via aiosqlite; full CRUD for alerts and capture sessions; aggregation queries used to power the dashboard's statistics.

- **main.py — FastAPI Orchestration**
  - Manages application startup and shutdown; exposes all REST endpoints with Pydantic validation; streams alerts in real time over a WebSocket; configures CORS for the dashboard.
- **dashboard/app.py — Streamlit SOC Dashboard**
  - A dark-themed operator interface with animated status indicators, interactive Plotly charts, an auto-refreshing alert feed, and a manual-analysis panel.

5.2 Technology Stack and Responsibilities

Component	Responsibility
Scapy	Captures and extracts network packets
Triage Engine	Quickly detects suspicious traffic patterns
ChromaDB / RAG	Retrieves relevant MITRE ATT&CK threat intelligence
GPT-OSS 120B (via Groq Cloud)	Classifies and explains flagged activity
FastAPI	Provides backend APIs and orchestrates the pipeline
SQLite	Stores alerts and capture sessions
Streamlit	Displays the real-time security dashboard
Nginx / Docker	Hosts the dashboard and API together under one deployment

5.3 REST API

Once the server is running, interactive Swagger documentation is available at http://localhost:8000/docs. The main endpoints exposed by the system are listed below.

Method	Endpoint	Description
GET	/	Welcome message and endpoint listing
GET	/status	System health for all components
GET	/interfaces	Lists available network interfaces

Method	Endpoint	Description
GET	/alerts	Paginated alerts with filters
GET	/alerts/recent	In-memory alert buffer (fast access)
GET	/alerts/{id}	Single alert detail
GET	/statistics	Aggregated data for dashboard charts
POST	/toggle-sniffing	Starts or stops the sniffer
POST	/analyze-sample	Manual packet analysis
WS	/ws/alerts	Real-time WebSocket alert stream

5.4 Dashboard and Alert Storage

The Streamlit dashboard shows the live status of the sniffer, the LLM client and the RAG engine, along with counts of packets captured and flagged, a breakdown of critical and high-severity alerts, threat-level and protocol distributions, an alert timeline, the most active source IP addresses, the attack vectors seen so far, and the full LLM explanation and recommended action for every alert. Every alert is written to SQLite together with its packet metadata, triage flags, LLM classification, confidence score, matched MITRE technique, explanation, recommended action and processing status, which together form a complete, queryable alert history.

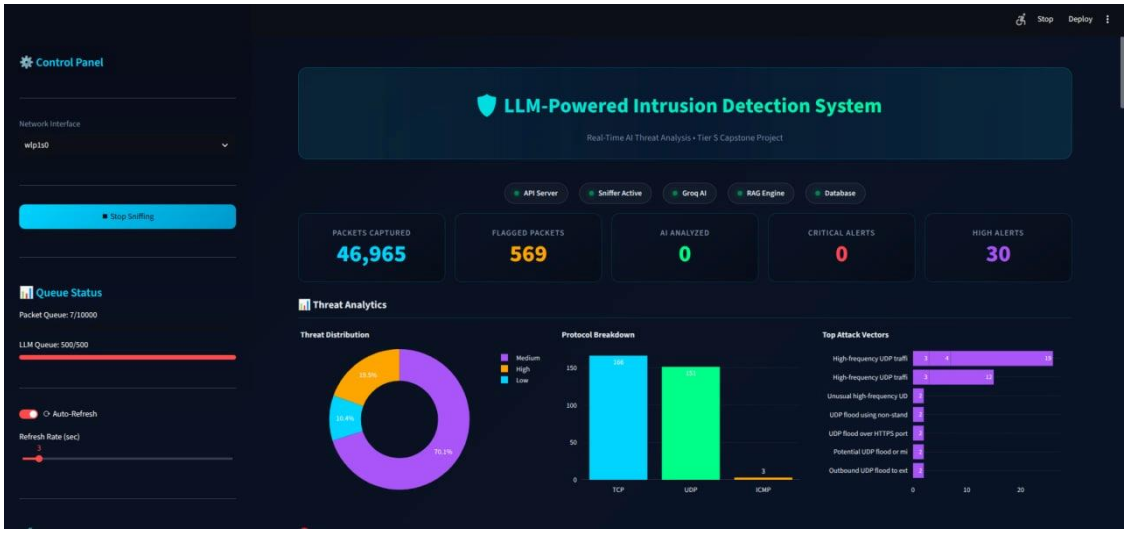

Figure 3: Live SOC dashboard, showing the control panel, system status, packet counters, threat distribution, protocol breakdown, and top attack vectors.

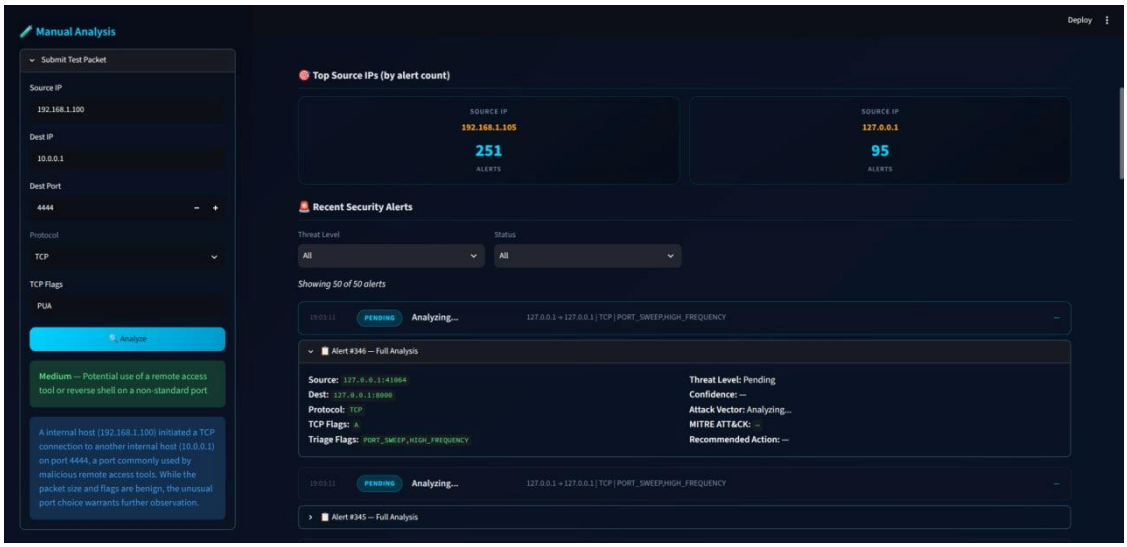

Figure 4: Manual analysis panel and recent security alerts, showing a submitted test packet and the corresponding LLM-generated alert with its threat level, confidence, MITRE mapping, and explanation.

6. Installation and Usage

6.1 Prerequisites

- Python 3.10 or newer
- Npcap (on Windows) — required for Scapy packet capture; install with "WinPcap API-compatible Mode" checked
- A Groq Cloud API key, used to access the GPT-OSS 120B model, set in the project's .env file
- Administrator or root privileges, required for raw packet capture

6.2 Setup Steps

- Navigate to the project directory (e.g. cd "IS Project")
- Create and activate a virtual environment: python -m venv venv, then venv\Scripts\activate
- Install dependencies: pip install -r requirements.txt
- Configure the .env file with the Groq API key and other settings
- Start the API server as Administrator: python main.py
- In a separate terminal, launch the dashboard: streamlit run dashboard/app.py

6.3 Operating the Dashboard

- Start / Stop Sniffing — a toggle button in the sidebar
- Interface Selection — choose the capture interface from a dropdown
- Manual Analysis — submit a test packet directly from the sidebar panel
- Auto-Refresh — toggle live updates, with a default 3-second interval

6.4 Generating Test Traffic

A few simple commands are enough to exercise the detection rules during a demonstration:

- Simple ICMP ping flood: ping -n 100 8.8.8.8
- Port scan with nmap: nmap -sS 192.168.1.1
- High-frequency connections (PowerShell): a loop of 200 Test-NetConnection calls against port 80

7. Limitations

- LLM analysis adds latency of roughly one to three seconds per flagged packet, because each classification requires a round trip to the Groq Cloud API.
- Packet capture requires Administrator or root privileges, which limits deployment in restricted environments.
- The current implementation captures from a single network interface only, with no multi-NIC aggregation.
- Groq's rate limits can throttle analysis during periods of very high suspicious-traffic volume.
- When the API and dashboard are deployed to a remote host such as Render, that host has no access to the user's local Wi-Fi traffic; a local sensor process is still required to capture packets and forward flagged metadata to the hosted API.

8. Future Work

- A voice-driven security assistant that can answer spoken queries about ongoing alerts.

- Multi-model comparison, evaluating GPT-OSS 120B against alternatives such as GPT-4 and Claude on the same traffic samples.
- PCAP file import and replay, to support offline analysis of previously captured traffic.
- Email and Slack notifications for high-severity alerts.
- GeoIP mapping to visualise the geographic origin of source and destination addresses.
- Integration with established SIEM platforms such as Splunk or the ELK stack.
- A fine-tuned, locally hosted model to allow fully offline operation without a cloud API dependency.

9. Conclusion

This project demonstrates that a hybrid approach to intrusion detection, combining fast deterministic heuristics with a language model grounded by Retrieval-Augmented Generation, can meaningfully close the gap between raw detection and usable, human-readable security insight. The heuristic triage engine keeps the system efficient by ensuring the language model only ever sees the traffic that genuinely warrants attention, while the RAG layer keeps the model's output tied to real MITRE ATT&CK knowledge rather than unguided generation.

The system is not intended to replace production-grade tools such as Snort or Suricata, and it does not attempt to match their throughput. Its value lies in the layer those tools do not provide: a plain-language explanation and a recommended next step for every alert, delivered through a live dashboard. As the future work items above are implemented, the system is well positioned to grow from an academic prototype into a genuinely useful SOC-augmentation tool.

10. Credits and Acknowledgements

- MITRE ATT&CK® — Threat-intelligence framework used for technique mapping
- GPT-OSS 120B, accessed via Groq Cloud — Large Language Model used for threat analysis
- Scapy — Packet manipulation and capture library

- FastAPI — Modern asynchronous web framework for the backend
- Streamlit — Framework used to build the SOC dashboard
- ChromaDB — Embedded vector database used for the RAG knowledge base